Parameter exploration of RQAOA on the Max-Cut problem

Luuk Daleman & Ivar van der Spoel

April 27, 2026

1 Introduction

Combinatorial optimization problems such as Max-Cut are interesting to both classical and quantum computing. The Quantum Approximate Optimization Algorithm (QAOA) is a prominent approach to these problems, but has limitations. Recursive Quantum Approximate Optimization Algorithm (RQAOA) was proposed to address these limitations by combining QAOA with a recursive low-depth strategy.

In this mini-project, we study RQAOA and explore different parameters. Focusing on the Max-Cut problem, we implement RQAOA on relatively small graph instances and analyze its approximation performance under different algorithmic settings. Furthermore, we explore if RQAOA also benefits from the parameter concentration phenomenon and how circuit depth impacts this property.

2 Background

In the background QAOA and RQAOA and explained, compared and the Max-k-Cut is explained. Moreover, the utility to realize this project is shared.

2.1 QAOA

Quantum Approximate Optimization Algorithm, QAOA for short, is a quantum algorithm designed to find approximate solutions to binary combinatorial optimization problems[FGG14], such as Max-Cut and Graph-coloring[HWO+19].

2.1.1 Description & strengths

QAOA represents the optimization problem as a quantum circuit consisting of alternating cost and mixer Hamiltonians. The cost Hamiltonian encodes the objective function, assigning higher amplitudes to better solutions, while the mixer Hamiltonian provides solutions to explore the search space. Each pair of cost and mixer Hamiltonians forms a layer, and applying multiple layers with optimized parameters γ and β for each of these Hamiltonians. These parameters are classically optimized with for example Gradient Descent or COBYLA [Pow94]. Eventually, this gradually increases the probability of measuring near-optimal solutions.

The main strengths of QAOA are:

- **Hybrid quantum-classical approach:** Both quantum and classical algorithms are utilized to find approximate solutions, with a classical optimizers and a quamtum circuit to find solutions.
- **Provable performance for small depths:** Even at low depth $p = 1$, QAOA can outperform random guessing for certain problems, like Max-Cut on specific graphs [FGG14].
- **Parameter tuning:** The classical optimization loop allows fine-tuning of parameters for specific instances, potentially improving approximation ratios over classical heuristics.

2.1.2 The no-go results

QAOA has limitations, often referred to as *no-go results*, for constant-depth QAOA on certain optimization problems. In particular, for problems such as Max-Cut and Max-k-Cut, QAOA at any fixed depth p **cannot** achieve approximation ratios comparable to the best classical algorithms [BKKT20, BKKT22].

These limitations come from the locality of low-depth QAOA circuits: correlations generated by the algorithm only optimizes in a local neighbourhood of solutions. As a result, QAOA behaves similarly to local classical algorithms and fails to capture the global structure required for good solutions. It has been shown that classical algorithms can **provably** outperform QAOA at any fixed depth. Using QAOA is *no-go* on these problems.

2.2 RQAOA

RQAOA, or Recursive Quantum Approximate Optimization Algorithm, expands on the QAOA concept. This method runs a relatively smaller QAOA circuit before "analysing" the results. Specifically, RQAOA uses constant-depth QAOA to identify which problem variables are most strongly correlated in the optimal solution[BGGS21].

2.2.1 Description & strengths

The algorithm proceeds as follows. First, QAOA is executed to obtain an optimal variational state. From this state, pairwise correlations between variables that interact in the cost function are estimated. The pair with the largest absolute correlation is then selected and a hard constraint is imposed: positively correlated variables are forced to take the same value, while negatively correlated variables are forced to take opposite values. This constraint removes one variable from the problem, reducing the instance size. The procedure is then repeated on the reduced problem.

This process continues until only n variables remain. The remaining (relatively small) optimization problem is then solved exactly using a classical brute-force solver. Finally, the full solution is reconstructed by applying the imposed constraints in reverse order.

The key strengths of RQAOA include:

- **Running-time and performance trade-off**: A hypothetical (on a quantum computer) running time speed-up is acquired with RQAOA, and with using a classical solver, the performance is considerably better than QAOA.
- **Constant-depth quantum circuits:** RQAOA typically relies on low-depth QAOA (often $p = 1$), making it well suited for noisy intermediate-scale quantum circuits unlike regular QAOA.
- **Improved scalability:** By recursively reducing the problem size, RQAOA mitigates the exponential growth of the Hilbert space encountered in standard QAOA.

2.2.2 Limitations

Despite its advantages, RQAOA also has some limitations:

- **Greedy variable elimination:** Once a constraint is imposed, it cannot be revised. Incorrect early decisions may cause poor solutions throughout the recursion.
- **Importance of capturing correlations:** The quality of the final solution strongly depends on how well low-depth QAOA captures the true correlations of the optimal solution.

2.3 Comparison

QAOA and RQAOA are closely related algorithms, yet they differ fundamentally in how quantum and classical resources are utilized. Standard QAOA attempts to solve the problem using a single deep quantum circuit, whose parameters are optimized and then measured to obtain a solution. While increasing the circuit depth p generally improves performance, constant-depth QAOA suffers from the aforementioned limitations on certain problem classes, including Max-Cut and Max-k-Cut, due to locality and scalability constraints [BKKT20, BKKT22].

RQAOA was introduced to overcome these limitations. Rather than relying on a single quantum circuit, RQAOA repeatedly executes smaller circuits with classical optimization. This recursive structure introduces non-local classical

optimization techniques between quantum steps, preventing the locality limitations that restrict standard QAOA [BKKT20].

Depth-1 RQAOA is particularly attractive for quantum devices, as it requires only shallow circuits and a small number of parameters. It has been shown to outperform depth-1 QAOA on the same instances [BKKT22].

2.4 Max-k-Cut Problem

The Max-k-Cut problem is a common combinatorial optimization problem defined on a weighted graph $G = (V, E)$. The goal is to split up the vertex set V into k disjoint subsets such that the total weight of edges connecting vertices in different subsets is maximized. The case where $k = 2$ is known as the *Max-Cut* problem.

For $k > 2$, Max-k-Cut generalizes this problem by allowing each vertex to take one of k labels. While QAOA can be extended to Max-k-Cut through encodings such as one-hot or clock representations, these extensions typically require additional qubits and more complex mixer Hamiltonians [HWO+19]. As a result, Max-k-Cut is a more challenging optimization task for near-term quantum algorithms.

In this work, we focus on the Max-Cut problem ($k = 2$).

2.5 Implementation details

All quantum algorithms were implemented using the `OpenQAOA` library [SSC+22]. Graph instances were generated and represented using `NetworkX`[HSS08], which integrates natively with OpenQAOA. Classical Max-Cut values for small graphs were computed via brute-force enumeration, enabling exact evaluation of approximation ratios.

3 Methodology

In this section we describe the experimental setup, implementation details, and the sequence of experiments used to evaluate the performance of RQAOA on the Max-Cut problem.

3.1 Approximation Ratio

Throughout all experiments, performance is quantified using the *approximation ratio*:

$$\alpha = \frac{C_{\text{alg}}}{C_{\text{opt}}},$$

where C_{alg} is the cut value obtained by the algorithm and C_{opt} is the optimal Max-Cut value. The optimal solution is obtained using a classical solver for all graphs. Do note that for much larger graphs there is a possibility to test using bipartite graphs, as $C_{\text{opt}} = |E|$ [BKKT20].

3.2 Graph families

Two kinds of random weighted graphs are constructed. The first is an Erdős–Rényi (ER) Graph with a 0.5 edge probability. The second is a 3-regular graph. Both are randomly generated with 16 nodes. Note that *#nodes* needs to be even with a 3-regular graph. Weights are randomly chosen to be between floats between 1 and 10. The motivation to use $n = 16$ nodes for all experiments is due to the computational complexity. $2^{16} = 65536$, so computationally checking each graph for the optimal solution is a realizable exercise for any classical machine.

For the first experiment in Section 3.3, Both types of graph are used. For the second experiment in Section 3.5 only the 3-regular graphs are used.

3.3 Parameter Configurations and Optimization

The purpose of this experiment is to investigate how the choice of classical optimizer, circuit depth p, and variable elimination cutoff n_c affects the approximation ratio of RQAOA on different graph families. We aim to identify configurations that give the best performance.

For this experiment, we consider five randomly generated instances of each graph family. We evaluate the following parameters:

- **Maximum iterations:** 200
- **Graph families:** ER ($n = 16, p_{\text{edge}} = 0.5$), 3-regular ($n = 16$)
- **Classical optimizers:** COBYLA, Powell, Nelder-Mead
- **Circuit depth:** $p \in \{1, 2, 3, 4\}$
- **Variable elimination cutoff:** $n_c \in \{1, 2, 3\}$
- **Step count:** $n_steps \in \{1, 2, 3\}$

Max iterations being the amount of iterations for the classical solver, and step count describing how many nodes get clumped up each recursion. We will be changing 1 parameter at a time. With the standard parameters being: COBYLA, $p = 1, n_c = 3, n_steps = 1$. Each combination of parameters is first optimized on each instance, and then the performance is measured in terms of the *approximation ratio*.

3.4 Running time

With the default parameters listed in the previous section, we will measure the running time of depth-1 RQAOA on the 5 instances for each graph family.

3.5 Parameter Concentration and Transferability

QAOA has a well known property of parameter concentration. For weighted MaxCut, this concentration extends when parameters β and γ are properly rescaled to account for edge weights [SLL+23]. This experiment investigates whether this phenomenon extends to RQAOA.

3.5.1 Method

Following the weighted MaxCut parameter transfer scheme, we account for both graph topology and edge weight magnitudes. For a graph G with average degree d and average absolute edge weight $|w| = \frac{1}{|E|}\sum_{(i,j)\in E}|w_{i,j}|$, parameters are scaled as:

$$(\beta^S, \gamma^S) = \left(\beta^*, \frac{\gamma^*}{\arctan(\sqrt{1/(d-1)})}\right)$$

where (β^*, γ^*) are the optimized parameters. To account for variability and random luck 30 training graphs and 15 test graphs were used.

In the **training phase**, we generate N_{train} weighted graphs, execute full RQAOA on each, scale the optimized parameters from all elimination steps, and compute median scaled parameters:

$$\tilde{\boldsymbol{\beta}}^S = \text{median}\{\boldsymbol{\beta}_j^S\}, \quad \tilde{\boldsymbol{\gamma}}^S = \text{median}\{\boldsymbol{\gamma}_j^S\}$$

In the **inference phase**, for a new weighted graph with average degree d_w and average absolute weight $|w|$, we transfer parameters via:

$$\beta_w = \tilde{\boldsymbol{\beta}}^S, \quad \gamma_w = \tilde{\boldsymbol{\gamma}}^S \cdot \frac{\arctan(\sqrt{1/(d_w-1)})}{|w|}$$

The rescaling by $|w|$ is critical: it accounts for the $\mathcal{O}(1/|w|)$ scaling of optimal γ values, enabling successful transfer across weighted instances.

Three strategies are compared: **direct transfer** (use (β_w, γ_w) without optimization), **warm-start** (initialize with (β_w, γ_w) and optimize), and **baseline** (random initialization). OpenQAOA allows for initial initialization of the β and γ, however, it is not possible to do so for every step in the RQAOA process. The optimizer is also called every iteration and is "disabled" by setting the max iteration to 0, effectively making it evaluation only. The warm-start version also sets the initial β and γ, but does not disable the optimizer. Lastly, a baseline RQAOA process is run with a random β and γ initialization. Depths 1 through 4 are tested and performance is measured via approximation ratio, see Section 3.1.

4 Results

In this section the results from the two experiments will be analyzed and discussed.

4.1 Parameter Configuration and Optimization results

The results can be observed in Figures 1, 2, 3 and 4. There are 5 instances for each graph family, each denoted with a separate seed number. Interestingly, for all observed graphs RQAOA performs considerably better on 3-regular graphs than on ER graphs. Different step sizes show no significant improvement in performance, but do converge to slightly different solutions. Notably, different cut-off points lead to exact same results, but in a few cases a different result. This is probably due to chance. However, increasing depth p demonstrates better approximations of the optimal solution. Different optimizers show no sign of a considerable advantage over another optimizer. However, across all tests, nelder-mead seems to perform slightly worsse than powell or cobyla.

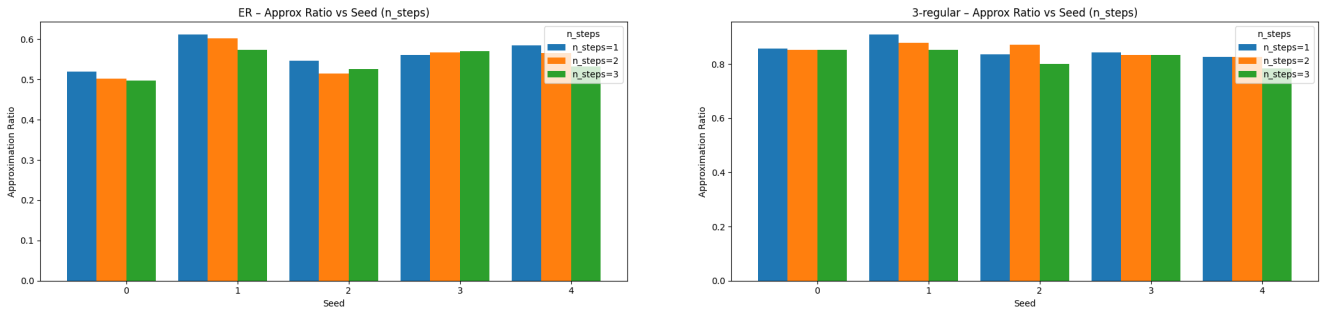

Figure 1: Different step sizes with the COBYLA optimizer on ER and 3-regular graphs on 5 different instances.

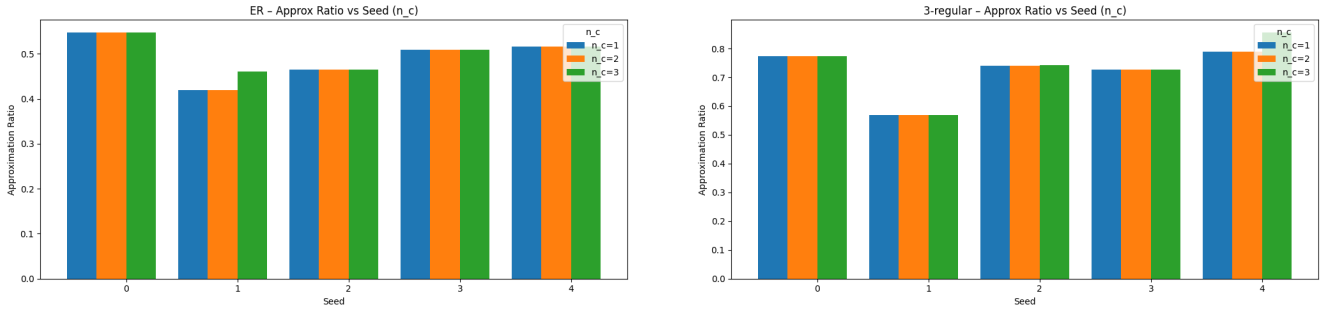

Figure 2: Different cut-off points with the COBYLA optimizer on ER and 3-regular graphs on 5 different instances.

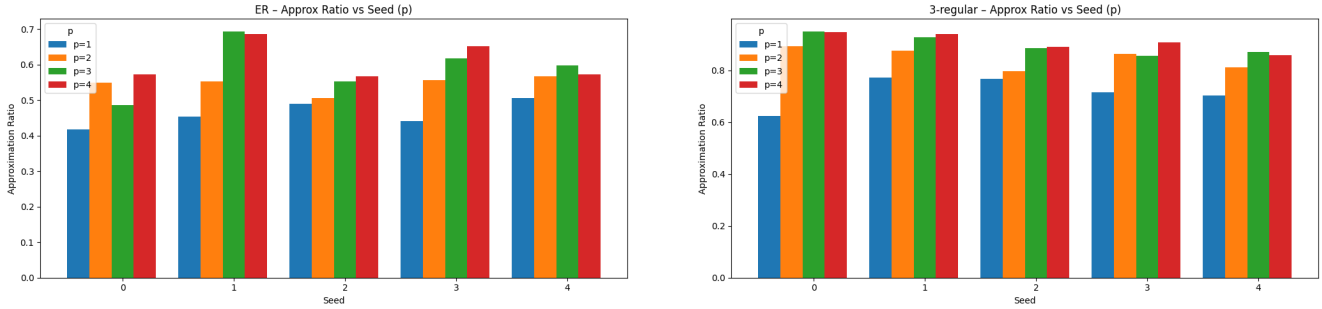

Figure 3: Different cut-off points with the COBYLA optimizer on ER and 3-regular graphs on 5 different instances.

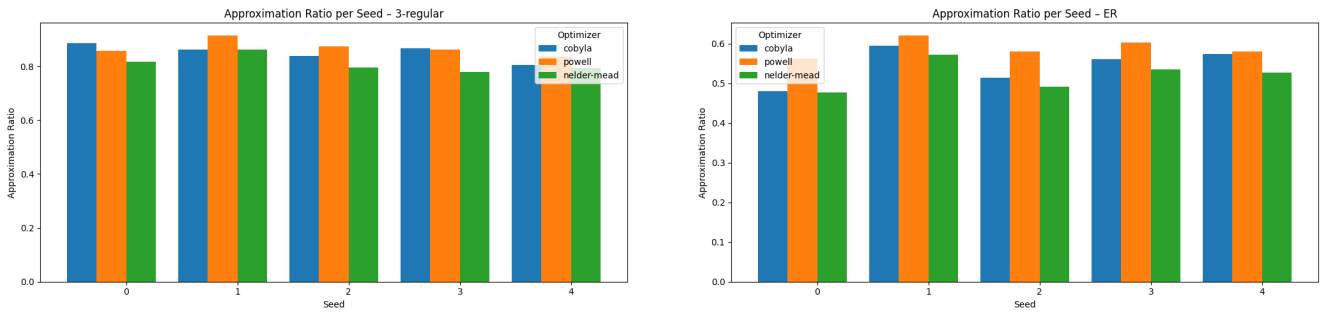

Figure 4: Different optimizers (with $p = 1$ and $n_c = 3$) on ER and 3-regular graphs on 5 different instances.

4.2 Running time

The running times for depth-1 RQAOA are presented in Table 1. The approximation ratios can be found in the previous section. The runtimes are always faster for 3-regular (as well as their approximation ratio being better).

Table 1: Execution times of depth-1 RQAOA with COBYLA and $n_c = 3$ for five random seeds on 3-regular and ER graphs.

Graph type	**Seed**	**Runtime (s)**
3-regular	0	8.52
3-regular	1	10.01
3-regular	2	6.76
3-regular	3	8.46
3-regular	4	7.25
ER	0	10.82
ER	1	17.89
ER	2	15.68
ER	3	10.70
ER	4	12.24

4.3 Parameter Concentration and Transferability

The results of the parameter transferability experiment are shown in Figure 5. These figures report the approximation ratios obtained over 4 circuit depths, using three strategies: direct transfer, warm-start transfer, and a baseline using random initialization.

The results show that the direct transfer strategy performs consistently poorly across all test instances. This suggests that the parameter values obtained during training do not directly generalize to new problem instances when applied without further optimization. In particular, the structural changes induced by the RQAOA variable elimination procedure appear to invalidate a naïve reuse of the learned parameters.

In contrast, the warm-start strategy achieves the highest median approximation ratio among the three methods in three out of four depths tested. This demonstrates that the trained parameters nevertheless encode useful information about the problem distribution, providing a systematically better initialization point for subsequent optimization than random initialization.

These observations indicate a transferable bias for the optimization process, but do not manifest a directly transferable policy. The limited performance of direct transfer may be attributed either to the strong instance dependence introduced by the recursive steps in RQAOA, or to limitations of the present transfer algorithm of the parameters.

As the circuit depth increases, the performance of the warm-start and random initialization strategies converges both in terms of median approximation ratio and variance. For larger depths, both methods consistently achieve

approximation ratios exceeding 80%. In contrast, the direct transfer strategy does not exhibit a systematic improvement with increasing circuit depth. Instead, its performance variability increases for depths greater than one. Moreover, the results for direct transfer are characterized by the presence of significant outliers. This suggests a strong sensitivity to instance-specific effects rather than reliable performance gains.

Lastly, the negative values are an artifact of the energy-to-objective conversion. Since MaxCut is encoded as a minimization of the negative cut value, a poor quantum state can yield a positive energy, which becomes negative after sign inversion. This indicates performance worse than a trivial baseline rather than an actual negative cut.

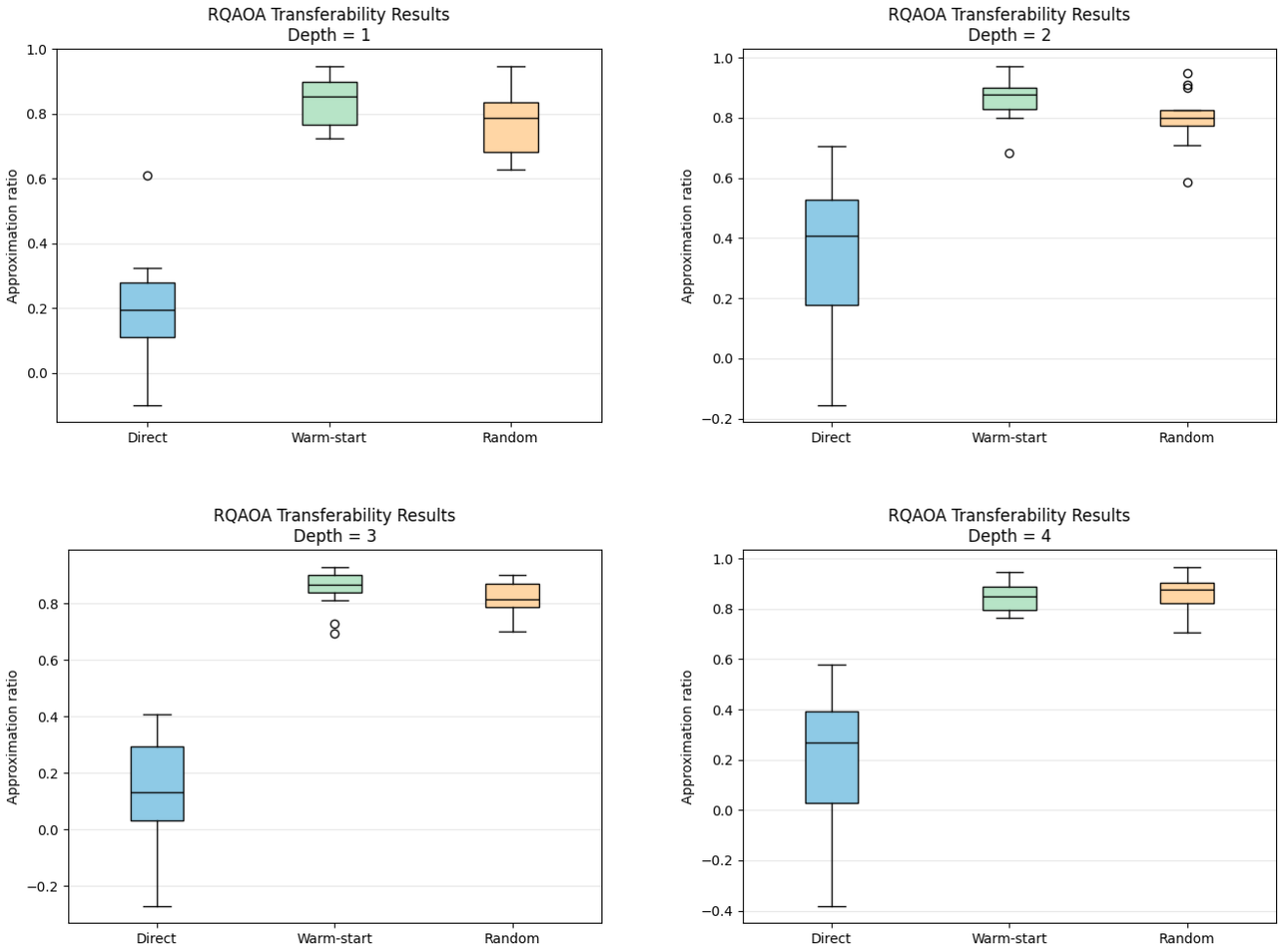

Figure 5: Different depths to visualize parameter transfer for Direct, Warm-start and Random.

5 Discussion

Due to the limitation of the openQAOA API we could not implement an algorithm that transfers parameters β and γ for every iteration of the algorithm. The warm-start strategy worked well, meaning our implementation may just not have been satisfactory to truly see good results. It would be interesting to see if this approach makes the direct transfer of the parameters perform well.

Increasing the QAOA depth generally leads to improved approximation results, as observed in Figure 3. It would be interesting to investigate the performance of deeper regular QAOA circuits as this would provide further insight in approximation quality regardless of the no-go results.

Only 5 instances were considered in the parameter configurations. Examining a larger set of instances could provide a more comprehensive understanding of performance trends. Despite the limited sample size, the results

suggests a ranking of optimizers, with Powell consistently outperforming COBYLA, and both outperform Nelder-Mead (Figure 4). It should be noted, however, that these findings may not generalize to other graph families.

No effect was measured when changing n_c, as shown in Figure 2. This is likely due to the classical component of the algorithm solving the subproblems, making the specific choice of n_c less critical. With significantly larger values of n_c, it could lead to reduced performance. This would be interesting to investigate. Importantly, a reduction in n_c would not result in a lower approximation ratio, consistent with the no-go result.

The running times reported in Table 1 indicate that 3-regular graphs are both faster to process and have better approximation ratios than ER graphs. These observations highlight the influence of graph structure on the efficiency and effectiveness of RQAOA.

6 Conclusion

In this mini-project, we implemented and analyzed the Recursive Quantum Approximate Optimization Algorithm (RQAOA) on the Max-Cut problem for small graph instances, with a focus on understanding the impact of various algorithmic parameters, including circuit depth, classical optimizer choice, variable elimination cutoff, and step size, as well as investigating parameter transferability.

3-regular graphs achieves higher approximation ratios compared to random graphs. Increasing the QAOA depth within RQAOA generally improves the solution quality, highlighting the benefits of deeper circuits when feasible. Among the classical optimizers tested, Powell and COBYLA performed slightly better than Nelder-Mead, although the differences were not substantial. Changes in variable elimination cutoff and recursion step size showed limited influence on performance for the small graphs considered, suggesting that the classical post-processing and recursion procedure dominate the final solution quality.

In contrast to standard QAOA, direct parameter transfer in RQAOA does not yield consistently strong performance. Nevertheless, the transferred parameters do provide a useful initialization for the parameters β and γ, as evidenced by the effectiveness of the warm-start strategy. Increasing the circuit depth does not improve the performance of direct transfer, whereas the warm-start and random initialization strategies converge to similar performance levels.

Overall, this project showcases that RQAOA is an interesting approach for combinatorial optimization problems. Combining shallow quantum circuits with classical solvers to minimize the limitations of standard QAOA, with potentially efficient and scalable on quantum computers benefiting from further parameter tuning and transfer strategies. Future work could explore larger graphs, different graph families, deeper circuits, different optimizers, and more advanced parameter transfer methods to better explore the known parameter concentration phenomenon within RQAOA and further investigate its potential advantage in quantum computing.

References

[BGGS21] Sergey Bravyi, David Gosset, Daniel Grier, and Luke Schaeffer. Classical algorithms for forrelation, 2021.

[BKKT20] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Obstacles to variational quantum optimization from symmetry protection. *Physical Review Letters*, 125(26), December 2020.

[BKKT22] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Hybrid quantum-classical algorithms for approximate graph coloring. *Quantum*, 6:678, March 2022.

[FGG14] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[HSS08] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In Gáel Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA, USA, August 2008.

[HWO+19] Stuart Hadfield, Zhihui Wang, Bryan O'gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

[Pow94] Michael JD Powell. A direct search optimization method that models the objective and constraint functions by linear interpolation. In *Advances in optimization and numerical analysis*, pages 51–67. Springer, 1994.

[SLL+23] Ruslan Shaydulin, Phillip C. Lotshaw, Jeffrey Larson, James Ostrowski, and Travis S. Humble. Parameter transfer for quantum approximate optimization of weighted maxcut. *ACM Transactions on Quantum Computing*, 4(3):1–15, April 2023.

[SSC+22] Vishal Sharma, Nur Shahidee Bin Saharan, Shao-Hen Chiew, Ezequiel Ignacio Rodríguez Chiacchio, Leonardo Disilvestro, Tommaso Federico Demarie, and Ewan Munro. Openqaoa – an sdk for qaoa, 2022.